

1. Завантаження аудіофайлу

- a. Ініціалізація завантаження: клієнт надсилає запит *POST /tracks/upload* із JWT-токеном через API GW до **Audio Metadata Service**.
- b. Перевірка лімітів: **Audio Metadata Service** робить gRPC-запит до **User Service**, який перевіряє в **PostgreSQL Users**, чи не перевищив користувач ліміт у 1000 треків.
- c. Генерація посилання: у разі успіху **Audio Metadata Service** створює запис про трек зі статусом *PENDING* у **PostgreSQL Tracks Metadata** та генерує тимчасове посилання *presigned URL* для завантаження в **AWS S3 Storage**.
- d. Пряме завантаження: клієнт завантажує оригінальний аудіофайл напряму в **AWS S3 Storage** за допомогою HTTP PUT запиту, обходячи backend-сервіси.
- e. Сповіщення про новий файл: після завершення завантаження **AWS S3 Storage** автоматично генерує подію *ObjectCreated Event* та надсилає її в чергу **AWS SQS**.
- f. Транскодування: **Audio Processing Worker** зчитує завдання з **AWS SQS**, завантажує сирий файл із S3 і за допомогою FFmpeg нарізає його на HLS-сегменти з різним бітрейтом та створює маніфест.
- g. Фіналізація: воркер зберігає готові HLS-сегменти назад у **AWS S3 Storage** та оновлює статус треку на *READY* у **PostgreSQL Tracks Metadata**.

2. Прослуховування аудіофайлу

- a. Запит на відтворення: клієнт робить запит *GET /tracks/{id}* через API GW до **Audio Metadata Service**.
- b. Отримання маніфесту: **Audio Metadata Service** перевіряє **Redis Cache** із фоллбеком на **PostgreSQL Tracks Metadata** у разі відсутності даних у кеші і повертає клієнту посилання на HLS-маніфест, який вказує на Content Delivery Network.
- c. Стримінг контенту: клієнтський плеєр запитує маніфест та перші аудіосегменти у CDN через HTTP Streaming.
- d. Кешування на межі: якщо сегмента немає в кеші CDN, edge-сервер завантажує його з **AWS S3 Storage** через origin request і кешує для наступних слухачів.
- e. Фіксація активності (heartbeats): під час відтворення клієнтський плеєр кожні 10-15 секунд надсилає фонові HTTP-запити (heartbeats) із даними про ID користувача, ID треку та тривалість прослуховування через API GW до **Analytics Ingestion Service**.

3. Поставлення «лайка» треку

- a. Запит на відмітку: користувач тисне «лайк», надсилаючи *POST /tracks/{id}/like* через API GW до **Likes Service**.
- b. Атомарний запис у кеш: **Likes Service** виконує атомарну операцію інкременту в **Redis Cache** (*INCR track:{id}:likes*) та фіксує факт лайка для користувача.
- c. Миттєва відповідь: сервіс відразу повертає відповідь *200 OK* клієнту. Завдяки зчитуванню лічильників з **Redis Cache** іншими користувачами, оновлена кількість лайків стає доступною аудиторії за 1-2 мілісекунди.
- d. Асинхронний фоновий скид: фоновий процес **Likes Service** періодично збирає накопичені зміни з **Redis** та збереженням пакетом синхронізує їх із **PostgreSQL Likes** для надійного довготривалого зберігання.

4. Перегляд статистики прослуховувань

a. Накопичення подій:

- i. **Analytics Ingestion Service** приймає потік heartbeats від плеєрів і без затримок перекидає їх у Apache Kafka.
- ii. **Analytics Processor** вичитає потік подій з Apache Kafka, у реальному часі групує їх за часовими вікнами (підраховуючи сумарну тривалість прослуховувань) та робить пакетні записи в **ClickHouse Analytical DB**.

b. Отримання аналітики користувачем

- i. Власник треків відкриває вкладку статистики, надсилаючи GET */users/me/stats* через API GW до **Statistics Service**.
- ii. **Statistics Service** робить оптимізований SQL-запит до **ClickHouse Analytical DB** із групуванням за часовими періодами.
- iii. ClickHouse агрегує мільйони записів у пам'яті за кілька мілісекунд і повертає підсумкові показники.
- iv. **Statistics Service** відправляє сформований JSON із готовою статистикою клієнту.